

Experiment No: 1
Date of Performance: 02-02-2026
Date of Submission: 07-02-2026
Student Name: Shreyans Tatiya
Roll No.: 16010123325
Batch: 3 Div: C
Staff In-charge:

TITLE: Encryption-Decryption using symmetric key

Objective:

To write a program for understanding Encryption-Decryption using classical cryptography (Playfair cipher/Substitution cipher Caesar cipher/affine cipher, Transposition cipher/rail fence cipher)

Expected OUTCOME of Experiment: Student will be able to

CO1: Explain various security goals, threats, vulnerabilities and controls

CO2: Apply various cryptographic algorithms for software security

Books/ Journals/ Websites referred:

1. Cryptography and Network Security, Behrouz.A. Fourouzan, Debdeep Mukhopadhyay, McGraw Hill Eighth Edition
 2. Cryptography and Network Security: Principles and Practice William Stallings, Pearson Eighth Edition
 3. Bernard Menezes Network Security and Cryptography, Cengage Learning Second Edition
-

Background Theory: (Answer in your handwriting)

Explain what is encryption and decryption with symmetric key cipher:

Q1. Encryption converts readable data into unreadable form using a key, and decryption follows the reverse process.

Encryption:

$$PT \rightarrow CT \quad \text{HELLO} \xrightarrow[\substack{\text{Encrypt} + \text{key}}]{+3} \text{KHOO R}$$

key = 3 (Caesar)

Decryption:

$$CT \rightarrow PT \quad \text{KHOO R} \xrightarrow[\substack{\text{Decrypt} + \text{key}}]{-3} \text{HELLO}$$

Explain Classical cryptography methods with example: Substitution / Transposition ciphers:

Q2. Classical Cryptography techniques:

- ① Substitution
- ② Transposition

→ Substitution

- ↳ each letter replaced by another letter.

eg: PT = HELLO $\xrightarrow{k=3}$ KHOO R [CAESAR CIPHER]

- ↳ simple to implement
- ↳ easy to break using frequency analysis.

→ Transposition

- ↳ letters are rearranged
- ↳ letters not changed
- ↳ preserves letter frequency

eg: [COLUMNAR] Keyless k=4

H	E	L	L	$\Rightarrow$	H	O	L	E	W	D	L	O	X	L	R	X
O	W	O	R													
L	D	X	X													

Playfair Cryptography with example:

Q3.

Playfair encrypts pair of letters (digraphs) instead of a single letter.

Rules:

- 5x5 key matrix
- I/J usually treated as one
- Encrypt text in letter pairs → in case of repeating fill in dummy.

eg: ~~key~~ ^{key =} MONARCHY

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Encrypt:

- ① Same row → right letter
- ② Same column → letter below
- ③ Rectangle → corner letters

PT = HELLO

HE → ~~CF~~ CF

LX → SU

LO → PM

⇒ CFSUPM

Laboratory Work:

Program Implementation with algorithm: 1) Substitution Cipher

Caesar Cipher

Code:

```
#include <iostream>
#include <string>
using namespace std;

string encrypt(string text, int shift) {
    string result = "";
    shift = shift % 26;

    for (char ch : text) {
        if (isupper(ch)) {
            char c = (ch - 'A' + shift) % 26 + 'A';
            result += c;
        }
        else if (islower(ch)) {
            char c = (ch - 'a' + shift) % 26 + 'a';
            result += c;
        }
        else {
            result += ch;
        }
    }
    return result;
}

string decrypt(string text, int shift) {
    return encrypt(text, 26 - (shift % 26));
}

int main() {
    string text;
    int shift;

    cout << "Enter text to encrypt: ";
    getline(cin, text);

    cout << "Enter shift value: ";
    cin >> shift;

    string encryptedText = encrypt(text, shift);
    cout << "Encrypted Text: " << encryptedText << endl;
```



```
string decryptedText = decrypt(encryptedText, shift);  
cout << "Decrypted Text: " << decryptedText << endl;
```

```
return 0;
```

Output:

```
Enter text to encrypt: Attack at Dawn
Enter shift value: 3
Encrypted Text: Dwwdfn dw Gdzq
Decrypted Text: Attack at Dawn
```

Monoalphabetic Cipher

Code:

```
import java.util.Scanner;

public class Monoalpha{

    public static String encrypt(String text) {
        String KEY = "QWERTYUIOPASDFGHJKLZXCVBNM";
        StringBuilder result = new StringBuilder();

        for (char ch : text.toCharArray()) {

            if (Character.isUpperCase(ch)) {
                result.append(KEY.charAt(ch - 'A'));
            }
            else if (Character.isLowerCase(ch)) {
                result.append(Character.toLowerCase(KEY.charAt(ch - 'a')));
            }
            else {
                result.append(ch);
            }
        }
        return result.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter plaintext: ");
        String text = sc.nextLine();

        System.out.println("Encrypted text: " + encrypt(text));
        sc.close();
    }
}
```

Output:

```
cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac Monoalpha.java && java Monoalpha
shreya@Shreyas-MacBook-Air-2 Lab % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac Monoalpha.java && java Monoalpha
Enter plaintext: hello WORLD!
Encrypted text: itssg VGKSR!
shreya@Shreyas-MacBook-Air-2 exp1 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac Monoalpha.java && java Monoalpha
Enter plaintext: THIS Code RUNS
Encrypted text: ZIOL Egrt KXFL
shreya@Shreyas-MacBook-Air-2 exp1 %
```


Polyalphabetic Cipher

Code:

```
import java.util.Scanner;

public class Polyalpha {

    // Encrypt
    public static String encrypt(String text, String key) {
        StringBuilder result = new StringBuilder();
        key = key.toUpperCase();
        int keyIndex = 0;

        for (char ch : text.toCharArray()) {

            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';

                int p = ch - base;
                int k = key.charAt(keyIndex % key.length()) - 'A';

                char c = (char) ((p + k) % 26 + base);
                result.append(c);

                keyIndex++;
            } else {
                result.append(ch);
            }
        }
        return result.toString();
    }

    // Decrypt
    public static String decrypt(String text, String key) {
        StringBuilder result = new StringBuilder();
        key = key.toUpperCase();
        int keyIndex = 0;

        for (char ch : text.toCharArray()) {

            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';

                int c = ch - base;
                int k = key.charAt(keyIndex % key.length()) - 'A';

                char p = (char) ((c - k + 26) % 26 + base);
                result.append(p);
            }
        }
        return result.toString();
    }
}
```



```
        keyIndex++;
    } else {
        result.append(ch);
    }
}
return result.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter plaintext: ");
    String text = sc.nextLine();

    System.out.print("Enter key: ");
    String key = sc.nextLine();

    String encrypted = encrypt(text, key);
    System.out.println("Encrypted Text: " + encrypted);

    System.out.println("Decrypted Text: " + decrypt(encrypted, key));
    sc.close();
}
```

Output:

```
cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac Polyalpha.java && java Polyalpha
shreya@Shreyas-MacBook-Air-2 Lab % cd "/Users/shreya/Desktop/CE/TY/Semest
er VI/NIS/Lab/exp1/" && javac Polyalpha.java && java Polyalpha
Enter plaintext: ATtack at Dawn
Enter key: Lemon
Encrypted Text: LXfopv ef Rnhr
Decrypted Text: ATtack at Dawn
shreya@Shreyas-MacBook-Air-2 exp1 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac Polyalpha.java && java Polyalpha
Enter plaintext: watch b99
Enter key: nine
Encrypted Text: jiggu j99
Decrypted Text: watch b99
shreya@Shreyas-MacBook-Air-2 exp1 %
```

Program Implementation with algorithm: 2) Transposition Cipher

Railfence

Code:

```
import java.util.*;

public class railfence {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the text :- ");
        String text = sc.nextLine();

        System.out.print("Enter the height value:- ");
        int height = sc.nextInt();

        encryption(text, height);
    }

    public static void encryption(String text, int height) {

        String en_text = "";
        boolean direction = false;
        int column = text.length();
        int row = 0;

        char[][] a = new char[height][column];

        for (int j = 0; j < column; j++) {
            a[row][j] = text.charAt(j);

            if (row == 0 || row == height - 1)
                direction = !direction;

            row += direction ? 1 : -1;
        }

        for (int i = 0; i < height; i++) {
            for (int j = 0; j < column; j++) {
                if (a[i][j] != '\u0000')
                    en_text += a[i][j];
            }
        }
    }
}
```

```
    }  
}  
  
System.out.println("Encrypted text = " + en_text);  
decryption(en_text, height);  
}  
  
public static void decryption(String encrypted_text, int height) {  
    int column = encrypted_text.length();  
    char[][] a = new char[height][column];  
  
    boolean direction = false;  
    int row = 0;  
    for (int j = 0; j < column; j++) {  
        a[row][j] = '*';  
  
        if (row == 0 || row == height - 1)  
            direction = !direction;  
  
        row += direction ? 1 : -1;  
    }  
  
    int index = 0;  
    for (int i = 0; i < height; i++) {  
        for (int j = 0; j < column; j++) {  
            if (a[i][j] == '*' && index < column) {  
                a[i][j] = encrypted_text.charAt(index++);  
            }  
        }  
    }  
  
    String decrypted_text = "";  
    row = 0;  
    direction = false;  
    for (int j = 0; j < column; j++) {  
        decrypted_text += a[row][j];  
  
        if (row == 0 || row == height - 1)
```

```
direction = !direction;

row += direction ? 1 : -1;
}
System.out.println("Decrypted text = " + decrypted_text);
}
}
```

Output:

```
cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac railfence.java && java railfence
shreya@Shreyas-MacBook-Air-2 NIS % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac railfence.java && java railfence
Enter the text :- lab at dawn
Enter the height value:- 2
Encrypted text = lba ana tdw
Decrypted text = lab at dawn
shreya@Shreyas-MacBook-Air-2 exp1 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac railfence.java && java railfence
Enter the text :- at the house tomorrow
Enter the height value:- 3
Encrypted text = aho owttehuetmro sor
Decrypted text = at the house tomorrow
shreya@Shreyas-MacBook-Air-2 exp1 %
```

Single Columnar Transposition

Code:

```
import java.util.*;

public class single_col{

    public static String encrypt(String text, String k) {
        text = text.replace(" ", "").toUpperCase();
        k = k.toUpperCase();
        String keyUpper = k;

        int cols = keyUpper.length();
        int rows = (int) Math.ceil((double) text.length() / cols);

        char[][] matrix = new char[rows][cols];
        int index = 0;

        // Fill matrix row-wise
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (index < text.length())
                    matrix[i][j] = text.charAt(index++);
                else
                    matrix[i][j] = 'X';
            }
        }

        Integer[] order = new Integer[cols];
        for (int i = 0; i < cols; i++) order[i] = i;

        Arrays.sort(order, (a, b) -> keyUpper.charAt(a) - keyUpper.charAt(b));

        StringBuilder cipherText = new StringBuilder();
        for (int col : order) {
            for (int row = 0; row < rows; row++) {
                cipherText.append(matrix[row][col]);
            }
        }

        return cipherText.toString();
    }

    public static String encryptKeyless(String text, int cols) {
        text = text.replace(" ", "").toUpperCase();
        int rows = (int) Math.ceil((double) text.length() / cols);
```

```
char[][] matrix = new char[rows][cols];
int index = 0;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (index < text.length())
            matrix[i][j] = text.charAt(index++);
        else
            matrix[i][j] = 'X';
    }
}

StringBuilder cipherText = new StringBuilder();
for (int col = 0; col < cols; col++) {
    for (int row = 0; row < rows; row++) {
        cipherText.append(matrix[row][col]);
    }
}

return cipherText.toString();
}

public static String decrypt(String cipherText, String key) {
    String keyUpper = key.toUpperCase();

    int cols = keyUpper.length();
    int rows = cipherText.length() / cols;

    char[][] matrix = new char[rows][cols];

    Integer[] order = new Integer[cols];
    for (int i = 0; i < cols; i++) order[i] = i;

    Arrays.sort(order, (a, b) -> keyUpper.charAt(a) - keyUpper.charAt(b));

    int index = 0;

    for (int col : order) {
        for (int row = 0; row < rows; row++) {
            matrix[row][col] = cipherText.charAt(index++);
        }
    }
}
```

```
StringBuilder plainText = new StringBuilder();
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        plainText.append(matrix[i][j]);
    }
}

return plainText.toString();
}

public static String decryptKeyless(String cipherText, int cols) {
    int rows = cipherText.length() / cols;

    char[][] matrix = new char[rows][cols];

    int index = 0;

    for (int col = 0; col < cols; col++) {
        for (int row = 0; row < rows; row++) {
            matrix[row][col] = cipherText.charAt(index++);
        }
    }

    StringBuilder plainText = new StringBuilder();
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            plainText.append(matrix[i][j]);
        }
    }

    return plainText.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter choice:\n1. Encrypt- Key\n2. Encrypt- Keyless");
    int choice = sc.nextInt();
    sc.nextLine();

    switch (choice) {
        case 1:
            System.out.println("Keyed Columnar Transposition");

            System.out.print("Enter message: ");
            String message = sc.nextLine();
        case 2:
            System.out.println("Keyless Columnar Transposition");

            System.out.print("Enter message: ");
            String message = sc.nextLine();
        default:
            System.out.println("Invalid choice");
    }
}
```



```
System.out.print("Enter key (word): ");
String k = sc.nextLine();

String encrypted = encrypt(message, k);
System.out.println("Encrypted message: " + encrypted);

String decrypted = decrypt(encrypted, k);
System.out.println("Decrypted message: " + decrypted);

break;

case 2:
    System.out.println("Keyless Columnar Transposition");

    System.out.print("Enter message: ");
    String mes = sc.nextLine();

    System.out.print("Enter number of columns: ");
    int cols = sc.nextInt();

    String encryptedKeyless = encryptKeyless(mes, cols);
    System.out.println("Encrypted message: " + encryptedKeyless);

    String decryptedKeyless = decryptKeyless(encryptedKeyless, cols);
    System.out.println("Decrypted message: " + decryptedKeyless);

    break;

default:
    System.out.println("Invalid choice");
    break;
}

sc.close();
}
```

Output(s):

```
cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac single_col.java && java single_col
shreya@Shreyas-MacBook-Air-2 NIS % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac single_col.java && java single_col
Enter choice:
1. Encrypt- Key
2. Encrypt- Keyless
1
Keyed Columnar Transposition
Enter message: meet me at dawn
Enter key (word): wow
Encrypted message: EMTWMTAAEEDN
Decrypted message: MEETMEATDAWN
shreya@Shreyas-MacBook-Air-2 exp1 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac single_col.java && java single_col
Enter choice:
1. Encrypt- Key
2. Encrypt- Keyless
2
Keyless Columnar Transposition
Enter message: attack at dawn
Enter number of columns: 3
Encrypted message: AAAATCTWTKDN
Decrypted message: ATTACKATDAWN
shreya@Shreyas-MacBook-Air-2 exp1 %
```

Double Columnar Transposition

```
import java.util.*;

public class double_col{

    public static String encrypt(String text, String k) {
        text = text.replace(" ", "").toUpperCase();
        k = k.toUpperCase();
        String keyUpper = k;

        int cols = keyUpper.length();
        int rows = (int) Math.ceil((double) text.length() / cols);

        char[][] matrix = new char[rows][cols];
        int index = 0;

        // Fill matrix row-wise
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (index < text.length())
                    matrix[i][j] = text.charAt(index++);
                else
                    matrix[i][j] = 'X';
            }
        }

        Integer[] order = new Integer[cols];
        for (int i = 0; i < cols; i++) order[i] = i;

        Arrays.sort(order, (a, b) -> keyUpper.charAt(a) - keyUpper.charAt(b));

        StringBuilder cipherText = new StringBuilder();
        for (int col : order) {
            for (int row = 0; row < rows; row++) {
                cipherText.append(matrix[row][col]);
            }
        }

        return cipherText.toString();
    }

    public static String encryptKeyless(String text, int cols) {
        text = text.replace(" ", "").toUpperCase();
        int rows = (int) Math.ceil((double) text.length() / cols);
```

```
char[][] matrix = new char[rows][cols];
int index = 0;

// Fill matrix row-wise
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (index < text.length())
            matrix[i][j] = text.charAt(index++);
        else
            matrix[i][j] = 'X';
    }
}

// Read column-wise (no sorting needed for keyless)
StringBuilder cipherText = new StringBuilder();
for (int col = 0; col < cols; col++) {
    for (int row = 0; row < rows; row++) {
        cipherText.append(matrix[row][col]);
    }
}

return cipherText.toString();
}

public static String decrypt(String cipherText, String key) {
    String keyUpper = key.toUpperCase();

    int cols = keyUpper.length();
    int rows = cipherText.length() / cols;

    char[][] matrix = new char[rows][cols];

    Integer[] order = new Integer[cols];
    for (int i = 0; i < cols; i++) order[i] = i;

    Arrays.sort(order, (a, b) -> keyUpper.charAt(a) - keyUpper.charAt(b));

    int index = 0;

    for (int col : order) {
        for (int row = 0; row < rows; row++) {
            matrix[row][col] = cipherText.charAt(index++);
        }
    }
}
```

```
StringBuilder plainText = new StringBuilder();
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        plainText.append(matrix[i][j]);
    }
}

return plainText.toString();
}

public static String decryptKeyless(String cipherText, int cols) {
    int rows = cipherText.length() / cols;

    char[][] matrix = new char[rows][cols];

    int index = 0;

    for (int col = 0; col < cols; col++) {
        for (int row = 0; row < rows; row++) {
            matrix[row][col] = cipherText.charAt(index++);
        }
    }

    StringBuilder plainText = new StringBuilder();
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            plainText.append(matrix[i][j]);
        }
    }

    return plainText.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter choice:\n1. Encrypt- Key\n2. Encrypt- Keyless");
    int choice = sc.nextInt();
    sc.nextLine();

    switch (choice) {
        case 1:
            System.out.println("Keyed Columnar Transposition");

            System.out.print("Enter message: ");
            String message = sc.nextLine();
        case 2:
            System.out.println("Keyless Columnar Transposition");

            System.out.print("Enter message: ");
            String message = sc.nextLine();
        default:
            System.out.println("Invalid choice");
    }
}
```

```
System.out.print("Enter key (word 1): ");
String k1 = sc.nextLine();

String inter = encrypt(message, k1);
System.out.println("Encrypted message: " + inter);

System.out.print("Enter key (word 2): ");
String k2 = sc.nextLine();

String encrypted = encrypt(inter, k2);
System.out.println("Double encrypted message: " + encrypted);

String decrypted = decrypt(encrypted, k2);
decrypted = decrypt(decrypted, k1);
System.out.println("Double decrypted message: " + decrypted);

break;

case 2:
    System.out.println("Keyless Columnar Transposition");

    System.out.print("Enter message: ");
    String mes = sc.nextLine();

    System.out.print("Enter number of columns: ");
    int cols = sc.nextInt();

    String inters = encryptKeyless(mes, cols);
    String encryptedKeyless = encryptKeyless(inters, cols);
    System.out.println("Encrypted message: " + encryptedKeyless);

    String X = decryptKeyless(encryptedKeyless, cols);
    String decryptedKeyless = decryptKeyless(X, cols);

    System.out.println("Decrypted message: " + decryptedKeyless);

    break;

default:
    System.out.println("Invalid choice");
    break;
}

sc.close();
}
```

Output(s):

```
shreya@Shreyas-MacBook-Air-2 exp1 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac double_col.java && java double_col
Enter choice:
1. Encrypt- Key
2. Encrypt- Keyless
2
Keyless Columnar Transposition
Enter message: meet me at dawn
Enter number of columns: 2
Encrypted message: MMDEEAEAWTTN
Decrypted message: MEETMEATDawn
shreya@Shreyas-MacBook-Air-2 exp1 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac double_col.java && java double_col
Enter choice:
1. Encrypt- Key
2. Encrypt- Keyless
2
Keyless Columnar Transposition
Enter message: attack at dawn
Enter number of columns: 4
Encrypted message: AKWCAADTTTAN
Decrypted message: ATTACKATDawn
shreya@Shreyas-MacBook-Air-2 exp1 %
```

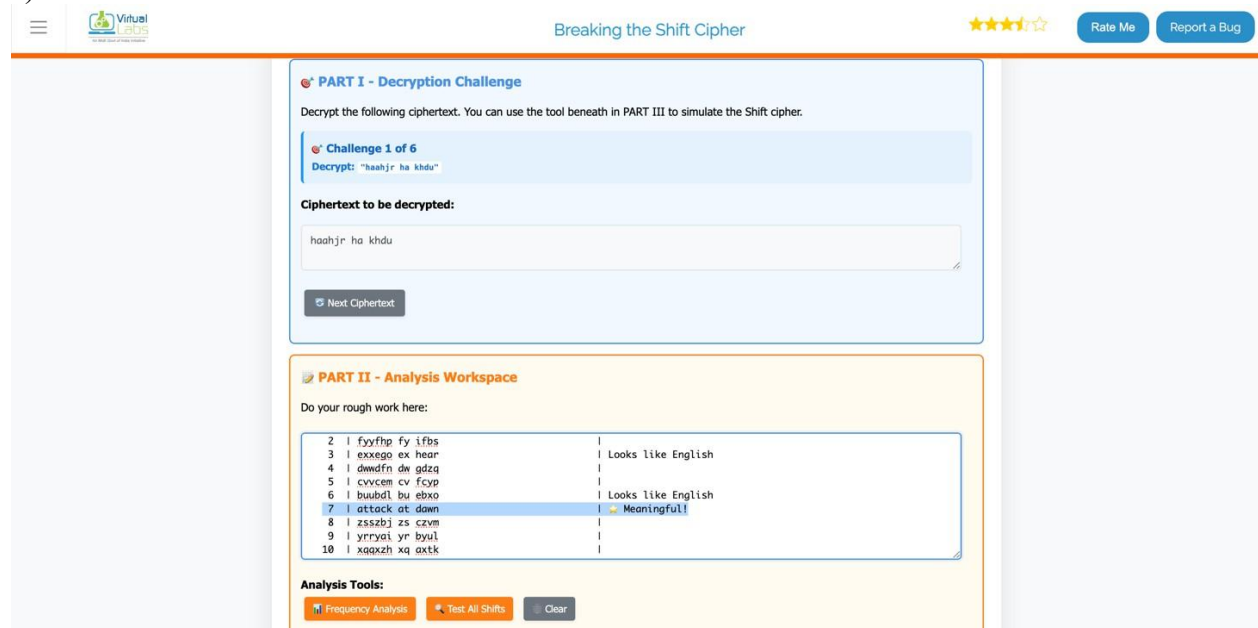
Post Lab Questions:

1. Encryption Decryptions (with screen shots of virtual lab)

Virtual Laboratory Experiments- (<http://cse29-iiith.vlabs.ac.in/>) :

(a). Breaking the Shift Cipher, (b). Breaking the Mono-alphabetic Substitution Cipher

a)



The screenshot shows the 'Breaking the Shift Cipher' virtual lab interface. The top navigation bar includes the 'Virtual Labs' logo, the title 'Breaking the Shift Cipher', a star rating, and buttons for 'Rate Me' and 'Report a Bug'.

PART I - Decryption Challenge

Decrypt the following ciphertext. You can use the tool beneath in PART III to simulate the Shift cipher.

Challenge 1 of 6
Decrypt: "haahjr ha khdu"

Ciphertext to be decrypted:

haahjr ha khdu

Next Ciphertext

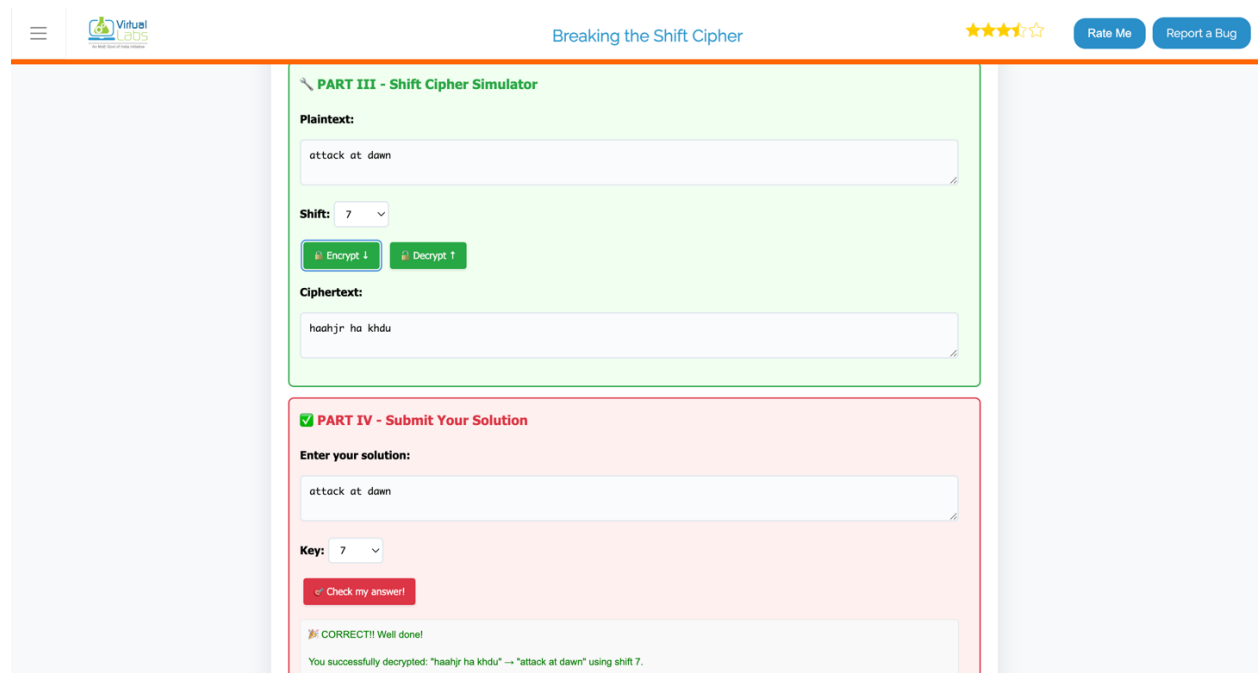
PART II - Analysis Workspace

Do your rough work here:

2		fyfthp	fy	ifbs		
3		exxego	ex	hear		Looks like English
4		dmdfn	dw	gdzq		
5		cvcem	cv	fcyp		
6		buubdl	bu	ebxo		Looks like English
7		attack	at	dawn		Meaningful!
8		zsszsj	zs	cvm		
9		yryai	yr	byul		
10		xqxzh	xq	atkl		

Analysis Tools:

Frequency Analysis Test All Shifts Clear



The screenshot shows the 'Breaking the Shift Cipher' virtual lab interface, continuing from Part II.

PART III - Shift Cipher Simulator

Plaintext:

attack at dawn

Shift: 7

Encrypt Decrypt

Ciphertext:

haahjr ha khdu

PART IV - Submit Your Solution

Enter your solution:

attack at dawn

Key: 7

Check my answer!

CORRECT!! Well done!

You successfully decrypted: "haahjr ha khdu" → "attack at dawn" using shift 7.

b)

 **Breaking the Mono-alphabetic Substitution Cipher** ★★★★☆ [Rate Me](#) [Report a Bug](#)

Part 2: Interactive Substitution Workspace

Substitution Scratchpad

Note: Cipher text is in *lower case*. When you replace characters, the final plaintext characters will be shown in *upper case* automatically.

Working Area:
Make substitutions and track your progress

Substitution Tools

Replace cipher characters with plaintext:
This function is case insensitive and replaces cipher text (lower case) with plain text (upper case):

Replace cipher: with plaintext: [Apply](#)

Undo substitutions (case sensitive):
Use this function to undo unwanted changes by specifying exact characters:

Replace text: with text: [Replace](#)

Replacement History

- You replaced ip oy tn
- You replaced rt by BY
- You replaced pxdxdxp by REMEMBER
- You replaced elxd by POEM
- You replaced idolokndu by COMPOUNDED
- You replaced bfarbwitt by INABILITY
- You replaced buxfjitt by IDENTITY
- You replaced ahat by AWAY
- You replaced rxnrxp by BEFORE
- You replaced crnhwfm by CRAWLING
- You replaced lxxwz by TELLS

ALICE COMES UPON A MUSHROOM AND SITTING ON IT IS A BLUE CATERPILLAR SMOKING A HOOKAH. THE CATERPILLAR QUESTIONS ALICE AND SHE ADMITS TO HER CURRENT IDENTITY CRISIS, COMPOUNDED BY HER INABILITY TO REMEMBER A POEM. BEFORE CRAWLING AWAY, THE CATERPILLAR TELLS ALICE THAT ONE SIDE OF THE MUSHROOM WILL MAKE HER TALLER AND THE OTHER SIDE WILL MAKE HER SHORTER. SHE BREAKS OFF TWO PIECES FROM THE MUSHROOM. ONE SIDE MAKES HER SHRINK SMALLER THAN EVER, WHILE ANOTHER CAUSES HER NECK TO GROW HIGH INTO THE TREES, WHERE A PIGEON MISTAKES HER FOR A SERPENT. WITH SOME EFFORT, ALICE BRINGS HERSELF BACK TO HER USUAL HEIGHT. SHE STUMBLES UPON A SMALL ESTATE AND USES THE MUSHROOM TO REACH A MORE APPROPRIATE HEIGHT.

 **Breaking the Mono-alphabetic Substitution Cipher** ★★★★☆ [Rate Me](#) [Report a Bug](#)

Part 3: Solution Verification

Submit Your Solution

After analyzing the ciphertext and making substitutions in the workspace above, enter your final solution below:

[Copy from Scratchpad](#)

Final Plaintext:
Enter the complete decrypted message

Solution Key:
26 letters (a-z), each exactly once

[Check Answer!](#)

CORRECT!! Well done!

Plaintext: alice comes upon a mushroom and sitting on it is a blue caterpillar smoking a hookah. the caterpillar questions alice and she admits to her current identity crisis, compounded by her inability to remember a poem. before crawling away, the caterpillar tells alice that one side of the mushroom will make her taller and the other side will make her shorter. she breaks off two pieces from the mushroom. one side makes her shrink smaller than ever, while another causes her neck to grow high into the trees, where a pigeon mistakes her for a serpent. with some effort, alice brings herself back to her usual height. she stumbles upon a small estate and uses the mushroom to reach a more appropriate height.

Key: ariuxnmebsgwdflloypzjkchvtq

Part 4: Encryption Tool



Breaking the Mono-alphabetic Substitution Cipher

★★★★☆[Rate Me](#)[Report a Bug](#)

[Check Answer!](#)

🎉 CORRECT!! Well done!

Plaintext: alice comes upon a mushroom and sitting on it is a blue caterpillar smoking a hookah. the caterpillar questions alice and she admits to her current identity crisis, compounded by her inability to remember a poem. before crawling away, the caterpillar tells alice that one side of the mushroom will make her taller and the other side will make her shorter. she breaks off two pieces from the mushroom. one side makes her shrink smaller than ever, while another causes her neck to grow high into the trees, where a pigeon mistakes her for a serpent. with some effort, alice brings herself back to her usual height. she stumbles upon a small estate and uses the mushroom to reach a more appropriate height.

Key: ariuxnmebsgwdffoyzjkhvtq

Part 4: Encryption Tool

🔧 Mono-Alphabetic Substitution Tool

Use this tool to encrypt/decrypt with your own key and plaintext:

Plaintext:
Your message to encrypt

defend the east wall of the castle

Substitution Key:
26-letter substitution alphabet

phgiuneaylnofdxjkrvcstzwb [Random Key](#)

[Encrypt](#) [Decrypt](#)

☐ Remove Punctuation

Ciphertext:
Encrypted result

giuifg cei iprc tpnn du cei qprcni

2. Explain Hill cipher with example and implement program.

Hill Cipher : Encryption.

Key = $\begin{bmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{bmatrix}_{3 \times 3}$ PT = POINT

$\begin{bmatrix} P & N \\ O & T \\ I & X \end{bmatrix}_{3 \times 2} \Rightarrow \begin{bmatrix} 15 & 13 \\ 14 & 19 \\ 8 & 23 \end{bmatrix}$

$\begin{bmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{bmatrix} \begin{bmatrix} 15 \\ 14 \\ 8 \end{bmatrix} \pmod{26} = \begin{bmatrix} 18 \\ 5 \\ 8 \end{bmatrix} = \begin{bmatrix} S \\ F \\ I \end{bmatrix}$

$\begin{bmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{bmatrix} \begin{bmatrix} 3 \\ 19 \\ 23 \end{bmatrix} \pmod{26} = \begin{bmatrix} 11 \\ 1 \\ 18 \end{bmatrix} = \begin{bmatrix} L \\ B \\ S \end{bmatrix}$

$\Rightarrow$ SFILBS

Code:

```
import java.util.*;

public class hill {
    private static final int MOD = 26;

    public static String encryptText(String plaintext, int[][] key) {
        plaintext = plaintext.toUpperCase().replaceAll(" ", "");
        int n = key.length;
        int padding = n - plaintext.length() % n;
        if (padding != n) {
            plaintext += "X".repeat(padding);
        }

        StringBuilder ciphertext = new StringBuilder();
        for (int i = 0; i < plaintext.length(); i += n) {
            int[] block = new int[n];
            for (int j = 0; j < n; j++) {
                block[j] = plaintext.charAt(i + j) - 'A';
            }
            int[] encryptedBlock = multiplyMatrix(key, block);
            for (int value : encryptedBlock) {
                ciphertext.append((char) (value + 'A'));
            }
        }
        return ciphertext.toString();
    }

    public static String decryptText(String ciphertext, int[][] key) {
        int determinant = determinant(key);
        int adjoint[][] = adjoint(key);
        int n = key.length;
        int[][] inverseKey = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                inverseKey[i][j] = (adjoint[i][j] * determinant) % MOD;
                if (inverseKey[i][j] < 0) {
                    inverseKey[i][j] += MOD;
                }
            }
        }
        return encryptText(ciphertext, inverseKey);
    }

    private static int[] multiplyMatrix(int[][] key, int[] block) {
        int n = key.length;
```

```
int[] result = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        result[i] += key[i][j] * block[j];
    }
    result[i] %= MOD;
}
return result;
}

private static int determinant(int[][] matrix) {
    if (matrix.length == 1) {
        return matrix[0][0];
    }
    int det = 0;
    for (int i = 0; i < matrix.length; i++) {
        int[][] minor = new int[matrix.length - 1][matrix.length - 1];
        for (int j = 1; j < matrix.length; j++) {
            for (int k = 0, col = 0; k < matrix.length; k++) {
                if (k == i) continue;
                minor[j - 1][col++] = matrix[j][k];
            }
        }
        det += Math.pow(-1, i) * matrix[0][i] * determinant(minor);
    }
    return det;
}

private static int[][] adjoint(int[][] matrix) {
    int n = matrix.length;
    int[][] adjoint = new int[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int[][] minor = new int[n - 1][n - 1];
            for (int k = 0, row = 0; k < n; k++) {
                if (k == i) continue;
                for (int l = 0, col = 0; l < n; l++) {
                    if (l == j) continue;
                    minor[row][col++] = matrix[k][l];
                }
                row++;
            }
            adjoint[i][j] = (int) Math.pow(-1, i + j) * determinant(minor);
        }
    }
    return transpose(adjoint);
}
```

```
private static int[][] transpose(int[][] matrix) {
    int[][] result = new int[matrix.length][matrix.length];
    for (int i = 0; i < matrix.length; i++) {
        for (int j = 0; j < matrix.length; j++) {
            result[i][j] = matrix[j][i];
        }
    }
    return result;
}

public static void main(String[] args) {
    int[][] key = {{6, 24, 1}, {13, 16, 10}, {20, 17, 15}};
    String plaintext = "POINT";
    String ciphertext = encryptText(plaintext, key);
    System.out.println("The Encrypted Text: " + ciphertext);
    String decrypted = decryptText(ciphertext, key);
    System.out.println("The Decrypted Text: " + decrypted);
}
```

Output:

```
cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac hill.java && java hill
shreya@Shreyas-MacBook-Air-2 NIS % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp1/" && javac hill.java && java hill
The Encrypted Text: SFILBS
The Decrypted Text: POINTX
shreya@Shreyas-MacBook-Air-2 exp1 %
```

Conclusion:

From this experiment, we understand the working of classical encryption and decryption techniques and their limitations. Although substitution and transposition ciphers provide basic confidentiality, they are not secure against modern attacks. The virtual experiments clearly demonstrated how easily classical ciphers can be broken, highlighting the importance of modern cryptographic algorithms for secure communication.